

3.3 Operations on Arrays

The basic operations in the Arrays are insertion, deletion, searching, display, traverse, and update. These operations are usually performed to either modify the data in the array or to report the status of the array.

Following are the basic operations supported by an array.

- **Traverse** – print all the array elements one by one.
- **Insertion** – Adds an element at the given index.
- **Deletion** – Deletes an element at the given index.
- **Search** – Searches an element using the given index or by the value.
- **Update** – Updates an element at the given index.
- **Display** – Displays the contents of the array.

1. Insertion Operation

- In the insertion operation, we are adding one or more elements to the array.
- Based on the requirement, a new element can be added at the beginning, end, or any given index of array.
- This is done using input statements of the programming languages.

Algorithm:

1. Start
2. Create an Array of a desired datatype and size.
3. Initialize a variable 'i' as 0.
4. Enter the element at ith index of the array.
5. Increment i by 1.
6. Repeat Steps 4 & 5 until the end of the array.
7. Stop

Example:

```
#include <stdio.h>

int main(){

int A[3] = {}, i;

printf("Array Before Insertion:\n");

for(i = 0; i < 3; i++)

printf("A[%d] = %d \n", i, A[i]);

printf("Inserting Elements.. \n");
```

```

printf("The array elements after insertion :\n"); // prints array values

for(i = 0; i < 3; i++) {

    A[i] = i + 2;

    printf("A[%d] = %d \n", i, A[i]);

}

return 0;

}

```

Output:

Array Before Insertion:

A[0] = 0

A[1] = 0

A[2] = 0

Inserting elements..

Array After Insertion:

A[0] = 2

A[1] = 3

A[2] = 4

A[3] = 5

A[4] = 6

2. Traversing Operation:

- Traversing an array means going through each element of an array exactly once.
- We start from the first element and go to the last element.

Algorithm:

- 1 Start
2. Initialize an Array of certain size and datatype.
3. Initialize another variable 'i' with 0.
4. Print the ith value in the array and increment i.
5. Repeat Step 4 until the end of the array is reached.
6. End

Example:

```
#include <stdio.h>
```

```
void main() {
```

```

int Arr[5] = {18, 30, 15, 70, 12};

int i;

printf("Elements of the array are:\n");

for(i = 0; i<5; i++) {

    printf("Arr[%d] = %d, ", i, Arr[i]);

}

}

```

```

Elements of the array are:
Arr[0] = 18, Arr[1] = 30, Arr[2] = 15, Arr[3] = 70, Arr[4] = 12,

```

3. Search operation:

- The search operation finds a particular data item or element in an array.
- We can search in an unsorted array with the help of traversal of the array.
- Searching an element in the array using a key; The key element sequentially compares every value in the array to check if the key is present in the array or not.

Algorithm

1. Start
2. Set J = 0
3. Repeat steps 4 and 5 while J < N
4. IF A[J] is equal ITEM THEN GOTO STEP 6
5. Set J = J +1
6. PRINT J, ITEM
7. Stop

Example:

```

#include <stdio.h>

void main(){

    int A[] = {1,3,5,7,8};

    int item = 5, n = 5;

    int i = 0, j = 0;

```

```

printf("The original array elements are :\n");

for(i = 0; i<n; i++) {

    printf("A[%d] = %d \n", i, A[i]);

}

for(i = 0; i<n; i++) {

    if( A[i] == item ) {

        printf("Found element %d at position %d\n", item, i+1);

    }

}

}

```

Output:

```

The original array elements are :
A[0] = 1
A[1] = 3
A[2] = 5
A[3] = 7
A[4] = 8
Found element 5 at position 3

```

4. Update Operation

- Update operation refers to updating an existing element from the array at a given index.

Syntax:

```
Arr_name[i]=new_value
```

Algorithm

1. Start
2. Set A[K-1] = ITEM
3. Stop

Example:

```

#include<stdio.h>

Int main()

```

```

{
int A[]={25,50,75,100,45};

A[3]=60;

printf("%d\n", A[3]);

Return 0;

}

```

Output :

60

5. Deletion Operation:

- In this array operation, we delete an element from the particular index of an array.
- This deletion operation takes place as we assign the value in the consequent index to the current index.

Algorithm

1. Start
2. Set J = K
3. Repeat steps 4 and 5 while J < N
4. Set A[J] = A[J + 1]
5. Set J = J+1
6. Set N = N-1
7. Stop

Example:

```

#include <stdio.h>

void main(){

    int A[] = {1,3,5};

    int n = 3;

    int i;

    printf("The original array elements are :\n");

    for(i = 0; i<n; i++)

```

```

    printf("A[%d] = %d \n", i, A[i]);

for(i = 1; i<n; i++) {

    A[i] = A[i+1];

    n = n - 1;

}

printf("The array elements after deletion :\n");

for(i = 0; i<n; i++)

    printf("A[%d] = %d \n", i, A[i]);

}

```

Output:

The original array elements are :

A[0] = 1

A[1] = 3

A[2] = 5

The array elements after deletion :

A[0] = 1

A[1] = 5

Advantages of Array in C

The following are the main advantages of an array:

1. Random and fast access of elements using the array index.
2. Use of fewer lines of code as it creates a single array of multiple elements.
3. Traversal through the array becomes easy using a single loop.
4. Sorting becomes easy as it can be accomplished by writing fewer lines of code.

Disadvantages of Array in C

1. Allows a fixed number of elements to be entered which is decided at the time of declaration.
Unlike a linked list, an array in C is not dynamic.
2. Insertion and deletion of elements can be costly since the elements are needed to be rearranged after insertion and deletion.

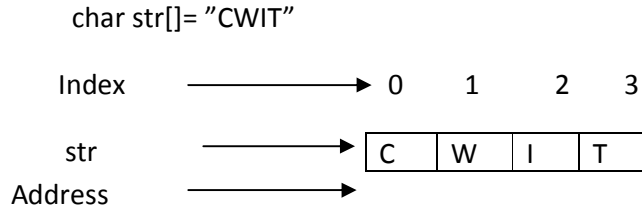
3.4 Character & string:

- String are typically stored as array of character, where each element represents a character in the sequence.
- Characters are stored as single element in memory.

Strings in C

- A String in C programming is a sequence of characters terminated with a null character `'\0'`.
- The C String is stored as an array of characters.
- The difference between a character array and a C string is that the string in C is terminated with a unique character `'\0'`.

Strings in C



Syntax:

`Data_type string_name [size];`

C String Initialization

- A string in C can be initialized in different ways.
- We will explain this with the help of an example.
- Below are the examples to declare a string with the name `str` and initialize it with `"CWIT"`.

We can initialize a C string in 4 different ways which are as follows:

1. Assigning a String Literal without Size

String literals can be assigned without size. Here, the name of the string `str` acts as a pointer because it is an array.

```
char str[] = "CWIT";
```

2. Assigning a String Literal with a Predefined Size

- String literals can be assigned with a predefined size.
- But we should always account for one extra space which will be assigned to the null character.
- If we want to store a string of size `n` then we should always declare a string with a size equal to or greater than `n+1`.

```
char str[50] = "CWIT";
```

3. Assigning Character by Character with Size

- We can also assign a string character by character.
- But we should remember to set the end character as `'\0'` which is a null character.

```
char str[6] = { 'C', 'W', 'I', 'T', '\0' };
```

4. Assigning Character by Character without Size

- We can assign character by character without size with the NULL character at the end.
- The size of the string is determined by the compiler automatically.

```
char str[6] = { 'C', 'W', 'I', 'T', '\0' };
```

Note: When a Sequence of characters enclosed in the double quotation marks is encountered by the compiler, a null character `'\0'` is appended at the end of the string by default.

C String Example

// C program to illustrate strings

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    // declare and initialize string
```

```
    char str[] = "CWIT";
```

```
    // print string
```

```
    printf("%s\n", str);
```

```
    int length = 0;
```

```
    length = strlen(str);
```

```
    // displaying the length of string
```

```
    printf("Length of string str is %d", length);
```

```
    return 0;
```

```
}
```

Output:

Length of string str is 5

1. Scanf() to read a string

```
#include<stdio.h>
```

```
Int main()
```

```
{
```

```
char name[20];
```

```
printf("Enter name:");
```

```
scanf("%s",name);
```

```
printf("Your name is %s",name);
```

```
return 0;
```

```
}
```


Output:

Enter name: CWIT

Your name is CWIT

2. fgets() & puts()

```
#include<stdio.h>

Int main()
{
char name[20];

printf("Enter name:");

fgets(name,sizeof(name),stdin);//read string

printf("Your name is ");

puts(name);

return 0;
}
```

Output:

Enter name: PUNE

Your name is PUNE

The C string functions are built-in functions that can be used for various operations and manipulations on strings. These string functions can be used to perform tasks such as string copy, concatenation, comparison, length, etc. The **<string.h>** header file contains these string functions.

In this article, we will discuss some of the most commonly used String functions in the C programming language.

String Functions in C

Some of the commonly used string functions in C are as follows:

1. strcat() Function

The strcat() function in C is used for string concatenation. It will append a copy of the source string to the end of the destination string.

Syntax:

char* **strcat**(char* *dest*, const char* *src*);

The terminating character at the end of **dest** is replaced by the first character of **src**.

Parameters

- **dest**: Destination string
- **src**: Source string

Return value

- The strcat() function returns a pointer to the dest string.

Example

// C Program to illustrate the strcat function

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char dest[50] = "This is an";
```

```
    char src[50] = " example";
```

```
    printf("dest Before: %s\n", dest);
```

```
    // concatenating src at the end of dest
```

```
    strcat(dest, src);
```

```
    printf("dest After: %s", dest);
```

```
    return 0;
```

```
}
```

Output:

dest Before: This is an

dest After: This is an example

2. strlen() Function

The strlen() function calculates the length of a given string. It doesn't count the null character '\0'.

Syntax:

```
int strlen(const char *str);
```

Parameters:

str: It represents the string variable whose length we have to find.

Return Value

strlen() function in C returns the length of the string.

Example:

```
// C program to demonstrate the strlen() function
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```

{
Char alphabet[]="ABCDEFGHIJKLMNOPQRSTUVWXYZ";

printf("Length is : %d\n",strlen(alphabet));

printf("Size is : %d\n",sizeof(alphabet));

return 0;

}

```

Output:

Length is: 26

Size is: 27

3. strcmp() Function

The strcmp() is a built-in library function in C. This function takes two strings as arguments and compares these two strings lexicographically.

Syntax:

```
int strcmp(const char *str1, const char *str2);
```

Parameters:

- **str1**: This is the first string to be compared.
- **str2**: This is the second string to be compared.

Return Value

- If str1 is less than str2, the return value is less than 0.
- If str1 is greater than str2, the return value is greater than 0.
- If str1 is equal to str2, the return value is 0.

Example:

// C program to demonstrate the strlen() function

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
Char str1[]="Hello";
```

```
Char str2[]="Hello";
```

```
Char str3[]="Hi";
```

```
//compare str1 & str2 and print the result
```

```
printf("%d\n",strcmp(str1,str2));

//compare str1 & str3 and print the result

printf("%d\n",strcmp(str1,str3));

return 0;

}
```

Output:

0

-4

4. strcpy Function

The strcpy() is a standard library function in C and is used to copy one string to another. In C, it is present in **<string.h>** header file.

Syntax

```
char* strcpy(char* dest, const char* src);
```

Parameters

- **dest**: Pointer to the destination array where the content is to be copied.
- **src**: string which will be copied.

Return Value

- strcpy() function returns a pointer pointing to the output string.

Example:

```
// C program to illustrate the use of strcpy()
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    // defining strings
```

```
    char source[] = "Hello world";
```

```
    char dest[20];
```

```
    // Copying the source string to dest
```

```
    strcpy(dest, source);
```

```
    // printing result
```

```
    printf("Source: %s\n", source);

    printf("Destination: %s\n", dest);

    return 0;

}
```

Output:

Hello world

Accepting string as the input

We have used scanf to accept the input from the user. However it can also be used in the case of string but with a different scenario.

Example:

```
#include <stdio.h>

int main()

{

    Char s[20];

    Printf("Enter the string:");

    Scanf("%s",s);

    Printf("You entered %s",s);

    return 0;

}
```

Output:

Enter string: CWIT Pune

You entered CWIT

- It is clear from the output that the above code will not work for separated string.
- To make this code working for space separated string the minor change required in the scanf function i.e instead of writing scanf("%s",s) we must write scanf("%[^\n]s",s) which instruct compiler to store the string s while new line(\n) encountered.

Example:

```
#include <stdio.h>

int main()
{
    Char s[20];

    Printf("Enter the string:");

    Scanf("[%^\n]s",s);

    Printf("You entered %s",s);

    return 0;
}
```

Output:

Enter string: CWIT Pune

You entered CWIT Pune

Passing string to function:

As strings are character array we pass string to function in the same way we pass an array to function.

```
#include<stdio.h>

Void printstr(char str[])
{
    Printf("string is:"%s",str);
}

Int main()
{
    Char str[]="CWIT"

    Printstr(str);

    Return 0;
}
```

Output:

String is : CWIT

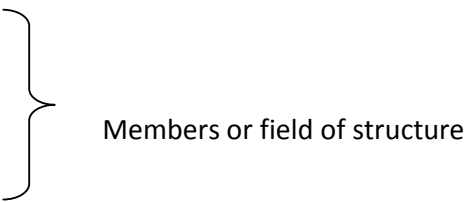
3.5 Structure

- Structure (also called struct) are a way to group several related variables into one place.
- Each variable in the structure is known as a member of the structure.
- Unlike array a structure can contains many different data types (int,float,char..etc)

Declaration and initialization of structure

Syntax:

```
Struct s_name
{
Datatype 1;
Datatype 2;
.
.
Datatype n;
} variable;
```



Members or field of structure

We have to declare structure in C before using it in our program. In structure declaration we specify its member variable along with their data type. We can use the struct keyword to declare the structure in C using the syntax.

Example:

```
Struct Employee
{
Char name[50];
Int id[5];
Float salary;
};
```

Array of Structures in C

- The array is a homogeneous collection of elements stored in the continuous memory location. The size of the array is fixed and we can randomly access the elements using their index.
- When dealing with a large set of related data and different data types, organizing and managing it efficiently is crucial. In C programming, the combination of arrays and structures i.e. array of structures provides a powerful tool for managing that.
- An array whose elements are of type structure is called array of structure. It is generally useful when we need multiple structure variables in our program.

Need for Array of Structures

Suppose we have 50 employees and we need to store the data of 50 employees. So for that, we need to define 50 variables of struct Employee type and store the data within that. However, declaring and handling the 50 variables is not an easy task. Let's imagine a bigger scenario, like 1000 employees. So, if we declare the variable this way, it's not possible to handle this.

```
struct Employee emp1, emp2, emp3, ... .. emp1000;
```

For that, we can define an array whose data type will be struct Employee so that will be easily manageable.

Declaration of Array of Structures

```
struct structure_name array_name [number_of_elements];
```

Initialization of Array of Structures

We can initialize the array of structures in the following ways:

```
1. struct structure_name array_name [number_of_elements] = {  
    {element1_value1, element1_value2, ....},  
    {element2_value1, element2_value2, ....},  
    .....  
    .....  
};
```

The same initialization can also be done as:

```
2. struct structure_name array_name [number_of_elements] = {  
    element1_value1, element1_value2 .....,  
    element2_value1, element2_value2 .....,  
};  
  
3. struct structure_name array_name [number_of_elements] = {  
    {.element3 = value, .element1 = value, ....},  
    {.element2 = value, .elementN = value, ....},  
    .....  
    .....  
};
```

Example:

```
#include <stdio.h>
```

```
struct student {  
  
    char firstName[50];  
  
    int roll;
```



```

float marks;

} s[5];

int main() {

    int i;

    printf("Enter information of students:\n");

    // storing information

    for (i = 0; i < 5; ++i) {

        s[i].roll = i + 1;

        printf("\nFor roll number%d,\n", s[i].roll);

        printf("Enter first name: ");

        scanf("%s", s[i].firstName);

        printf("Enter marks: ");

        scanf("%f", &s[i].marks);

    }

    printf("Displaying Information:\n\n");

    // displaying information

    for (i = 0; i < 5; ++i) {

        printf("\nRoll number: %d\n", i + 1);

        printf("First name: ");

        puts(s[i].firstName);

        printf("Marks: %.1f", s[i].marks);

        printf("\n");

    }

    return 0;

}

```

Output:

```
Enter information of students:
```

```
For roll number1,  
Enter name: Tom  
Enter marks: 98
```

```
For roll number2,  
Enter name: Jerry  
Enter marks: 89
```

```
.  
.
```

Displaying Information:

```
Roll number: 1  
Name: Tom  
Marks: 98
```

```
.  
.
```

3.6 C typedef

- The **typedef** is a keyword that is used to provide existing data types with a new name. The C typedef keyword is used to redefine the name of already existing data types.
- When names of datatypes become difficult to use in programs, typedef is used with user-defined datatypes, which behave similarly to defining an alias for commands.

C typedef Syntax

```
typedef existing_name alias_name;
```

After this declaration, we can use the *alias_name* as if it were the real *existing_name* in our C program.

Example:

```
// C program to implement typedef  
  
#include <stdio.h>  
  
// defining an alias using typedef  
typedef long long ll;  
  
// Driver code  
int main()  
{  
    // using typedef name to declare variable  
    ll var = 20;  
    printf("%ld", var);  
}
```

```
        return 0;
    }
}
```

Output:

20

typedef struct

typedef can also be used with structures in the C programming language. A new data type can be created and used to define the structure variable.

Example 1: Using typedef to define a name for a structure

```
// C program to implement
// typedef with structures

#include <stdio.h>

#include <string.h>

// using typedef to define an alias for structure
typedef struct students {
    char name[50];
    char branch[50];
    int ID_no;
} stu;

// Driver code

int main()
{
    stu st;

    strcpy(st.name, "Kamlesh Joshi");
    strcpy(st.branch, "Computer Science And Engineering");
    st.ID_no = 108;

    printf("Name: %s\n", st.name);
    printf("Branch: %s\n", st.branch);
    printf("ID_no: %d\n", st.ID_no);

    return 0;
}
```

```
}
```

Output:

Name: Kamlesh Joshi

Branch: Computer Science And Engineering

ID_no: 108

typedef with Pointers

- typedef can also be used with pointers as it gives an alias name to the pointers.
- Typedef is very efficient while declaring multiple pointers in a single statement because pointers bind to the right on the simple declaration.

Example:

```
typedef int* Int_ptr;
```

```
Int_ptr var, var1, var2;
```

In the above statement var, var1, and var2 are declared as pointers of type int which helps us to declare multiple numbers of pointers in a single statement.

```
// C program to implement
```

```
// typedef with pointers
```

```
#include <stdio.h>
```

```
typedef int* ptr;
```

```
// Driver code
```

```
int main()
```

```
{
```

```
    ptr var;
```

```
    *var = 20;
```

```
    printf("Value of var is %d", *var);
```

```
    return 0;
```

```
}
```

Enumeration (or enum) in C

Enumeration (or enum) is a user defined data type in C. It is mainly used to assign names to integral constants, the names make a program easy to read and maintain.

Example:

```
// An example program to demonstrate working
// of enum in C

#include<stdio.h>

enum week{Mon, Tue, Wed, Thur, Fri, Sat, Sun};

int main()
{
    enum week day;

    day = Wed;

    printf("%d",day);

    return 0;
}
```

Output:

2

In the above example, we declared “day” as the variable and the value of “Wed” is allocated to day, which is 2. So as a result, 2 is printed.